Desempenho da API Barberaria — Relatório

Data: 2026-10-14
Regra avaliada: toda API deve responder em **menos de 1 segundo**.
Escopo: todas as 64 rotas expostas por `apps/api` (NestJS).

Resumo

Resultado	Rotas
✅ Abaixo de 1s	52
⚠️ Abaixo de 1s isolada, mas acima de 1s com usuários simultâneos	3
❌ Acima de 1s (duas delas também falham com erro 500)	8
❔ Não medida	1

Problemas principais:

1. **Reativar profissional ou serviço:** a rota estoura o timeout de 5s da transação, devolve erro 500 e **não reativa o recurso**. É um bug funcional, além de lentidão.
2. **Listagens do Painel Master que juntam dados de todas as barbearias** (dashboard, atividade, usuários, assinaturas, pagamentos e repasses): o tempo cresce junto com o número de barbearias e passa de 1s com 55 barbearias.
3. **Login, cadastro e criação de profissional:** passam isolados (~0,5s), mas, como o hash de senha é feito com `bcryptjs`, os pedidos são atendidos em fila. Com 5 logins simultâneos, o tempo chega a ~1,7s.
4. **Região do banco:** se o Supabase estiver longe do Render (Oregon), por exemplo em São Paulo, até as rotas simples ficam perto de 1s.

---

1. Como os testes foram feitos

Ambiente

- API compilada (`npm run build`) e executada com `node dist/main.js`, `NODE_ENV=production`.
- PostgreSQL 16 local, com todas as migrations (`0001` – `0009`) aplicadas por `prisma migrate deploy`.
- A API usou a role restrita `barberaria_app` (`npm run db:setup-role`): `NOSUPERUSER` e `NOBYPASSRLS`, a mesma configuração de produção. Assim o RLS foi exercitado de verdade.
- Cache em memória (fallback do `CacheModule`, sem `REDIS_URL`).
- Máquina de 4 vCPUs, com Node 22.

Massa de dados

Toda a massa foi criada pela própria API, exceto o volume de agendamentos, inserido em lote via SQL:

Item	Quantidade
Barbearias (tenants)	55, cada uma com admin, assinatura, 6 pagamentos e 1 repasse

Item	Quantidade
Barbearia principal (`bench-0`)	12 barbeiros com horário de 08:00 às 20:00 nos 7 dias da semana, 10 serviços vinculados a todos os barbeiros e ~30 clientes
Agendamentos	~26 mil, cobrindo 300 dias passados e 60 dias futuros, 6 horários por dia por barbeiro
Agendamentos futuros	~370 por barbeiro e ~500 por serviço

Cenários de latência entre API e banco

Para simular a distância de rede entre o Render e o Supabase, foi usado um proxy TCP (`latproxy.mjs`) que atrasa cada pacote:

Cenário	Ida e volta (RTT)	Representa
Local	~0 ms	API e banco na mesma máquina; é o melhor caso
Produção estimada	**20 ms**	Render em Oregon (`render.yaml`) com Supabase na costa oeste dos EUA
Banco distante	170 ms	Render em Oregon com Supabase em São Paulo (`sa-east-1`); só algumas rotas foram verificadas

A regra de 1s foi julgada no cenário "Produção estimada" (20 ms).

Método de medição

- O tempo medido é o total da requisição HTTP vista pelo cliente, do envio até receber o corpo completo.
- Cada rota foi chamada 10 vezes em sequência (5 vezes para provisionar barbearia e para ativar/inativar). O valor usado é o **p95**: 95% das chamadas levaram até esse tempo.
- As rotas de escrita usam um registro novo em cada repetição (e-mail, slug, horário e agendamento distintos), para não medir só caminhos de erro.
- Ativar/inativar foi testado com o barbeiro e o serviço mais carregados: 370 e 504 agendamentos futuros, respectivamente.
- Teste de concorrência (`conc.mjs`): rajadas de 5 a 50 requisições simultâneas em login, disponibilidade, listagem de agendamentos e dashboard.

Scripts (versionados em `apps/api/scripts/benchmark/`)

Arquivo	Função
`bench.mjs`	Cria a massa de dados e mede todas as rotas
`conc.mjs`	Teste de concorrência
`latproxy.mjs`	Proxy de latência

> ⚠️ `bench.mjs` cria dezenas de tenants `bench-*`. **Não rode contra o banco de homologação/produção**. Veja o risco já registrado no `CLAUDE.md` sobre a suíte e2e compartilhar banco.

Reprodução resumida:

```
# Postgres local na porta 55432, com migrations e role aplicadas (ver apps/api/.env.example)
SEED_SUPER_ADMIN_EMAIL=sa@bench.com SEED_SUPER_ADMIN_PASSWORD=benchpass123 npm run seed:super-admin
node scripts/benchmark/latproxy.mjs 55433 55432 10 &   # RTT 20ms
# subir a API com DATABASE_URL apontando para a porta 55433
node scripts/benchmark/bench.mjs http://localhost:3000 50 10 resultado.json
node scripts/benchmark/conc.mjs
```

---

2. Resultado por API

Os tempos são p95, em milissegundos.

Autenticação e rotas públicas

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /health`	Health check	3	4	✅
`GET /v1/public/tenants/:slug`	Branding público do tenant	5	25	✅
`POST /v1/auth/login`	Login	357	474	⚠️ ok isolado, falha com concorrência (seção 3.3)
`POST /v1/auth/register`	Cadastro de cliente	341	524	⚠️ ok isolado, falha com concorrência (seção 3.3)
`POST /v1/auth/refresh`	Renova access token (cookie)	3	5	✅
`POST /v1/auth/logout`	Logout	8	2	✅

Usuário e tenant

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/users/me`	Perfil do usuário logado	8	118	✅
`GET /v1/users/:id`	Usuário por id	6	96	✅
`GET /v1/tenants/me`	Config/branding do próprio tenant	4	48	✅
`PATCH /v1/tenants/me`	Atualiza branding	7	51	✅
`POST /v1/tenants/me/logo/presign`	URL assinada para upload de logo	—	—	❔ não medido: depende do Supabase Storage, sem credenciais no ambiente de teste

Serviços

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/services`	Lista serviços (paginado)	10	161	✅
`GET /v1/services/:id`	Detalhe de serviço	9	115	✅
`POST /v1/services`	Cria serviço	8	121	✅
`PATCH /v1/services/:id`	Edita serviço	11	147	✅
`PATCH /v1/services/:id {active:false}`	Inativa serviço (504 agendamentos → `needs_reschedule`)	50	193	✅
`PATCH /v1/services/:id {active:true}`	**Reativa serviço** (reverte os agendamentos)	651	**5.026 → erro 500**	❌
`GET /v1/services/:id/professionals`	Profissionais que fazem o serviço	12	183	✅

Profissionais e horários

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/professionals`	Lista profissionais	15	159	✅
`GET /v1/professionals/:id`	Detalhe de profissional	15	94	✅
`POST /v1/professionals`	Cria profissional	351	475	⚠️ ok isolado, usa bcrypt (seção 3.3)
`PATCH /v1/professionals/:id`	Edita profissional	9	141	✅
`PATCH /v1/professionals/:id {active:false}`	Inativa profissional (370 agendamentos)	17	166	✅
`PATCH /v1/professionals/:id {active:true}`	**Reativa profissional**	469	**5.021 → erro 500**	❌
`GET /v1/professionals/:id/services`	Serviços do profissional	8	141	✅
`POST /v1/professionals/:id/services`	Vincula serviço	13	120	✅
`DELETE /v1/professionals/:id/services/:sid`	Desvincula serviço	7	96	✅
`GET /v1/professionals/:id/working-hours`	Lista horários de trabalho	7	94	✅
`POST /v1/professionals/:id/working-hours`	Cria horário	13	120	✅

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`PATCH /v1/professionals/:id/working-hours/:id`	Edita horário	9	119	✅
`DELETE /v1/professionals/:id/working-hours/:id`	Remove horário	6	117	✅

Agendamentos

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/professionals/:id/availability`	Horários livres do dia	17	186	✅
`POST /v1/appointments`	Cria agendamento	13	209	✅
`GET /v1/appointments` (cliente)	Histórico do cliente	16	121	✅
`GET /v1/appointments` (admin)	Agenda da barbearia	9	124	✅
`GET /v1/appointments?from&to`	Agenda da semana	9	180	✅
`GET /v1/appointments/:id`	Detalhe de agendamento	5	94	✅
`PATCH /v1/appointments/:id/cancel`	Cancela	10	122	✅
`PATCH /v1/appointments/:id/complete`	Conclui (admin)	8	123	✅
`PATCH /v1/appointments/:id/reschedule`	Remarca	11	186	✅

Painel Master (super_admin)

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/admin/whoami`	Identidade do super admin	2	2	✅
`GET /v1/admin/dashboard/overview`	KPIs do painel	95	**2.187**	❌
`GET /v1/admin/activity`	Atividade recente	40	**1.033**	❌
`GET /v1/admin/users`	Usuários de todos os tenants	56	**1.125**	❌
`GET /v1/admin/subscriptions`	Assinaturas de todos os tenants	41	**1.415** sem cache / 2 com cache	❌
`GET /v1/admin/payments`	Pagamentos de todos os tenants	31	**1.133** sem cache / 2 com cache	❌
`GET /v1/admin/payouts`	Repasses de todos os tenants	32	**1.107** sem cache / 2 com cache	❌
`GET /v1/admin/tenants`	Lista barbearias	7	50	✅
`GET /v1/admin/tenants/:id`	Detalhe barbearia	4	48	✅
`POST /v1/admin/tenants`	Provisiona barbearia, admin e assinatura	358	658	✅
`PATCH /v1/admin/tenants/:id`	Suspende/reativa barbearia	11	166	✅
`GET /v1/admin/plans`	Catálogo de planos	4	47	✅

API	Funcionalidade	Local	Prod. (20ms)	< 1s?
`GET /v1/admin/plans/:id`	Detalhe de plano	3	26	✅
`POST /v1/admin/plans`	Cria plano	6	—	✅ (medido só no local)
`PATCH /v1/admin/plans/:id`	Edita plano	5	74	✅
`GET /v1/admin/tenants/:id/subscription`	Assinatura de um tenant	10	156	✅
`PATCH /v1/admin/tenants/:id/subscription`	Troca plano	13	339	✅
`POST /v1/admin/tenants/:id/payments`	Registra pagamento	13	167	✅
`PATCH /v1/admin/tenants/:id/payments/:id`	Marca pagamento pago/pendente	13	182	✅
`POST /v1/admin/tenants/:id/payouts`	Registra repasse	11	161	✅
`PATCH /v1/admin/tenants/:id/payouts/:id`	Marca repasse pago/pendente	10	182	✅
`GET /v1/admin/settings`	Configurações da plataforma	4	48	✅
`PATCH /v1/admin/settings`	Edita configurações	6	71	✅

---

3. Análise dos problemas

3.1 Reativar profissional ou serviço: bug, além de lentidão

- **Onde:** `apps/api/src/common/revert-needs-reschedule.util.ts`.
- **Causa:** a função faz um `UPDATE` por agendamento, linha a linha, dentro da transação interativa do Prisma. Cada linha custa uma ida e volta ao banco, e a transação tem timeout padrão de 5s.
- **Efeito:** com 20 ms de latência, 370 agendamentos já ultrapassam 5s. A rota devolve **500**, a transação inteira é desfeita e **o profissional ou serviço continua inativo**, com os agendamentos ainda em `needs_reschedule`. O admin fica sem conseguir reativar.
- **Limite prático:** para ficar abaixo de 1s com RTT de 20 ms, cabem ~45 agendamentos futuros. No local, o mesmo erro aconteceu com ~4.400.
- **Direção de correção:** resolver a reversão com um único `UPDATE ... WHERE NOT EXISTS (conflito)` em SQL. Mantém a mesma regra (não sobrescrever um horário já ocupado, índice da seção 5.3) e faz uma só ida ao banco.

3.2 Agregações do Painel Master crescem com o número de barbearias

- **Onde:** `admin-dashboard.service.ts`, `admin-activity.service.ts`, `admin-users.service.ts`, `admin-subscriptions.service.ts`, `admin-payments.service.ts`, `admin-payouts.service.ts`.

- **Causa:** essas tabelas têm RLS forçado e a role de runtime é `NOBYPASSRLS`. Por isso a soma é feita abrindo **uma transação por barbearia** (`runInTenantContext`), no máximo 5 em paralelo (`mapWithConcurrency`). O custo cresce na mesma proporção do número de barbearias.
- **Efeito:** com 55 barbearias, entre 1,0s e 2,2s. Com 3 chamadas simultâneas ao dashboard, ~3,6s cada. Assinaturas, pagamentos e repasses têm cache de 90s, então só a chamada que encontra o cache vazio é lenta. Dashboard, atividade e usuários não têm cache, por decisão já documentada.
- **Direções de correção** (exigem decisão, porque mexem na estratégia de RLS do Super Admin):
  - Fazer cada transação por tenant juntar suas várias consultas em uma só ida ao banco.
  - Aumentar a concorrência, respeitando o pool.
  - Criar uma view ou função `SECURITY DEFINER` só de leitura agregada para o Super Admin. Isso seria uma exceção nova de RLS e precisa de aprovação explícita.

3.3 Login, cadastro e criação de usuário sob concorrência

- **Causa:** o `bcryptjs` (JavaScript puro) com custo 12 gasta ~330ms de CPU por hash, na thread principal do Node. As requisições simultâneas acabam enfileiradas.
- **Medição** (`POST /v1/auth/login`, RTT 20 ms):

Logins simultâneos	p50	máx
5	1.683ms	1.685ms
10	3.272ms	3.275ms
20	3.973ms	5.892ms

- O plano `starter` do Render tem menos CPU que a máquina de teste, então o tempo tende a ser pior em produção.
- **Direções de correção:**
  - Trocar `bcryptjs` por `bcrypt` (nativo, roda fora da thread principal) ou `argon2`. Ambos estão de acordo com a seção 6.1.4 do `CLAUDE.md`.
  - Avaliar o custo 10–11 em vez de 12.

3.4 Concorrência nas rotas comuns está ok

Com RTT de 20 ms, as rotas comuns ficaram abaixo de 1s mesmo com 50 requisições simultâneas:

Rota	10 simultâneas	20 simultâneas	50 simultâneas
`GET /availability`	p50 204 / máx 339	p50 269 / máx 401	p50 424 / máx 816
`GET /appointments`	p50 165 / máx 275	p50 231 / máx 347	p50 356 / máx 689

3.5 A região do banco é decisiva

Cada rota de tenant faz ~4–5 idas e voltas ao banco: abre a transação, executa o `set_config` (RLS, seção 6.3), faz a consulta e fecha a transação. Esse custo é esperado e não deve ser removido (seção 8.6). Por isso a distância entre API e banco pesa muito.

Com o banco em São Paulo e a API em Oregon (RTT ~170 ms):

Rota	Tempo
`GET /v1/users/me`	702ms
`GET /v1/services`	871ms
`GET /v1/appointments`	873ms
`GET /v1/professionals/:id/availability`	1.048ms ❌
`POST /v1/auth/login`	1.852ms ❌

Recomendação: confirmar a região do projeto Supabase e colocar API (Render) e banco (Supabase) na mesma região. É a ação de maior impacto e não exige mudança de código.

---

4. Conclusão

- **O núcleo do MVP atende à regra de 1s** no cenário de produção estimado: agendamento, disponibilidade, histórico, CRUDs de serviços, profissionais e horários, e branding, com folga, entre ~100 e 210ms.
- **Não atendem à regra:**
    1. ❌ Reativar profissional ou serviço: além de lento, **falha com 500 e não reativa**. É prioridade alta.
    2. ❌ Seis listagens agregadas do Painel Master: passam de 1s com ~55 barbearias e pioram conforme o número cresce.
    3. ⚠️ Login, cadastro e criação de profissional: passam isolados, mas estouram 1s com 5 ou mais pedidos simultâneos.
- **Pré-condição de infraestrutura:** API e banco na mesma região. Sem isso, até as rotas simples ficam perto de 1s.
- **Ordem sugerida de correção:**
    1. Reativação em lote (bug).
    2. Região do Supabase.
    3. Troca do `bcryptjs`.
    4. Agregações do Painel Master (precisa decidir a estratégia de RLS).